

Homework 2

Data Analysis and Machine Learning with Python

學號 B12502174

姓名許珈寧

Problem description:

There are two datasets and the corresponding questions. Each dataset is 50 points. If you use AI tool(s) to help this assignment, **please tell me why you selected that/those AI tool(s) and attach screenshots or a webpage of your conversation with the AI tool(s)**. Each question requires an explanation of your thoughts and actions, along with relevant models, evidence or charts if possible. Additionally, not all questions have standard answers.

Note: same as Homework 1, you need to prepare a report recording all ideas, steps, processes, results, and so on of your answers for the questions.

Dataset 1 descriptions (maintenance_prediction.csv):

A company operates a fleet of devices that transmit daily sensor readings. They aim to develop a predictive maintenance solution to anticipate when maintenance should be conducted. This approach offers potential cost savings compared to routine or time-based preventive maintenance, as tasks are only performed when necessary.

The objective is to construct a predictive model using machine learning to predict the probability of device failure. When building this model, it's crucial to minimize false positives and false negatives. The target variable to predict is labeled "failure," with binary values of 0 for non-failure and 1 for failure.

Column descriptions

date: the data received day

device: device ID

failure: device can work or not, 0: workable/non-failure, 1: failure

metric1~9: features

(10%) Q1. How many unique device IDs are there in this dataset?

直接在 **device** 的欄位統計 **unique** 的個數，總共有 1169 個。

Q1. How many unique device IDs are there in this dataset?

```
[7]: unique_value=df['device'].nunique()  
unique_value
```

```
[7]: 1169
```

(10%) Q2. You are asked to do data analysis. What key findings can you expect to uncover?

會希望透過訓練集的資料來發現特徵一到九與是否失敗的規律，進而在測試集有好的效果

(15%) Q3. You are asked to build a prediction model (with default parameters' settings). Which kind of machine learning will be used and why? Supervised learning or Unsupervised learning? Regression, classification, or clustering?

You can choose one or more models from the following models: Single Linear Regression, Multiple Linear Regression, Polynomial Regression, Logistic Regression, KNN, SVM, Gaussian Naive bayes, Multinomial Naive bayes, Decision Tree, Random Forest, XGBoost, KMeans, DBSCAN, and AgglomerativeClustering.

我會選擇使用監督式學習因為這份資料是已經有標籤，並且使用分類，因為這是一個二維的離散數據，根據上課所學以及網路上所查到的資料分類法會是這類數據最好的模型。

儘管如此我還是有同時用回歸的方法來訓練模型，想看看是否真的使用分類的效果會比較好以下是我選擇的模型，以及他們的程式碼。

1. Logistic Regression

```
from sklearn.linear_model import LogisticRegression
x = df.iloc[:,2:10].values
y = df.iloc[:,1].values

clf = LogisticRegression()
clf.fit(x,y)
```

2. Polynomial regression

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
poly_reg = PolynomialFeatures(degree = 2)

x_poly = poly_reg.fit_transform(x)
reg2 = LinearRegression()

reg2.fit(x_poly, y)
```

3. SVC

```
from sklearn import svm
clf = svm.SVC()
clf.fit(x,y)
```

4. K-NN

```
from sklearn import neighbors
clf = neighbors.KNeighborsClassifier()
clf.fit(x,y)
```

5. Gaussian Naive bayes

```
from sklearn.naive_bayes import GaussianNB
clf = GaussianNB()
clf.fit(x,y)
```

6. Multinomial Naive bayes

```
from sklearn.naive_bayes import MultinomialNB
clf = MultinomialNB()
clf.fit(x,y)
```

7. Decision tree regression

```
from sklearn.tree import DecisionTreeRegressor
clf=DecisionTreeRegressor()
clf.fit(x,y)
```

8. DecisionTreeClassifier

```
from sklearn import tree
clf = tree.DecisionTreeClassifier()
clf.fit(x,y)
```

9. RandomForestClassifier

```
from sklearn.ensemble import RandomForestClassifier
clf=RandomForestClassifier()
clf.fit(x,y)
```

10. XGBClassifier

```
from xgboost.sklearn import XGBClassifier
clf = XGBClassifier()
clf.fit(x,y)
```

我分別讓每一個模型都以測試集其中之一的成功和失敗的產品來作比較，並且再把測試集全部拿進去測來看準確率

以下是我最終跑出來的結果

	0	1	Accuracy
Logistic Regression	0	1	0.9991164232814432
Polynomial Regression	0.0005194	0.02058938	0.02541832214801687
SVC	0	0	0.9991485533439363
K-NN	0	0	0.9991485533439363
Gaussian Naïve bayes	0	1	0.9974054974536926
MultinomialNB	0	1	0.724717657075843
DecisionTreeRegression	0	1	0.9952789991875695
<i>Decision Tree</i>	0	1	0.9999919674843768
<i>Random forest</i>	0	1	0.9999919674843768
<i>XGBoost</i>	0	1	0.9996546018282005

可以發現用分類法所測出來的結果較使用回歸法更為準確，至少到小數點下三位都為九，根據準確度，我會選擇

Decision Tree, Random forest, XGBoost 來作為第四題的解題工具。

(15%) Q4. There are two data from two devices, please predict the corresponding failure values.

	metric								
device	1	2	3	4	5	6	7	8	9
Test1	127 175 526	410 9.43 4	3.90 566	54.6 320 8	15.4 622 6	258 303. 5	30.6 226 4	30.6 226 4	23.0 849 1
Test2	452 737 6	0	0	0	3	24	0	0	0

由第三題可以知道，分類法在判斷這類資料有更高的準確度因此我們將測試集的資料放進去三個模型中跑，由下表可以發現跑出來的結果皆為零，代表 **device** 都是正常的。

	<i>Decision Tree</i>	<i>Random forest</i>	<i>XGBoost</i>
Test1	0	0	0
Test2	0	0	0

下面的圖片是我所跑的程式供參考：

Q4. There are two data from two devices, please predict the corresponding failure values.

```
Test1=[127175526,4109.434,3.90566,54.63208,15.46226,258303.5,30.62264,23.08491]
Test2=[4527376,0,0,0,3,24,0,0]
```

1.Decision Tree

```
# Decision Tree
from sklearn import tree
clf = tree.DecisionTreeClassifier()
clf.fit(x,y)
print(clf.predict([[127175526,4109.434,3.90566,54.63208,15.46226,258303.5,30.62264,23.08491]]))
print(clf.predict([[4527376,0,0,0,3,24,0,0]]))
```

```
[0]
```

```
[0]
```

Random Forest

```
#Random forest
from sklearn.ensemble import RandomForestClassifier
clf=RandomForestClassifier()
clf.fit(x,y)
print(clf.predict([[127175526,4109.434,3.90566,54.63208,15.46226,258303.5,30.62264,23.08491]]))
print(clf.predict([[4527376,0,0,0,3,24,0,0]]))
```

```
[0]
```

```
[0]
```

XGBoost

```
# XGBoost
from xgboost.sklearn import XGBClassifier
clf = XGBClassifier()
clf.fit(x,y)
print(clf.predict([[127175526,4109.434,3.90566,54.63208,15.46226,258303.5,30.62264,23.08491]]))
print(clf.predict([[4527376,0,0,0,3,24,0,0]]))
```

```
[0]
```

```
[0]
```

AI 的使用

HW1 與 AI 的對話在下面連結

<https://chatgpt.com/share/67efd215-0dc4-800d-a5cf-b02f1307fda2>

基本上我是在 Jupyter notebook 上如果跑不出來的話會把該區段的程式碼以及錯誤訊息直接貼到 ChatGPT 上請他幫我解釋，通常解釋完之後程式就能夠跑了

Dataset 2 descriptions (feedback_sentiment.csv):

This dataset comprises customer sentiments expressed across various sources such as social media, review platforms, testimonials, and more. It includes text, sentiment (positive or negative), source of the sentiment, date/time of the sentiment, user ID, location, and confidence score. The sentiments reflect customers' opinions and experiences with products, services, movies, music, books, restaurants, websites, customer support, and more.

You can perform the following tasks on the dataset:

Data cleaning

Sentiment analysis

Statistical analysis

Data visualization

Text preprocessing

Topic modeling

Feature engineering

Machine learning modeling

Data integration

Data aggregation

Note: there is only one column in the csv file. You must use the separator argument within Pandas to read the data.

(4%) Q1. How many columns are there in this csv file and what are these columns' names?

總共有七個，分別是 Text, Sentiment, Date/Time, User ID, Location, confidence score

```
df.head()
```

	Text, Sentiment, Source, Date/Time, User ID, Location, Confidence Score
0	"I love this product!", Positive, Twitter, 202...
1	"The service was terrible.", Negative, Yelp Re...
2	"This movie is amazing!", Positive, IMDb, 2023...
3	"I'm so disappointed with their customer suppo...
4	"Just had the best meal of my life!", Positive...

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 98 entries, 0 to 97
Data columns (total 1 columns):
#   Column                                     Non-Null Count  Dtype
---  ---
0    Text, Sentiment, Source, Date/Time, User ID, Location, Confidence Score  96 non-null     object
dtypes: object(1)
memory usage: 916.0+ bytes
```

```
# 進行拆分
df.columns = ['raw_data']
df[['text', 'sentiment', 'source', 'date/time', 'user_id', 'location', 'confidence_score']] = df['raw_data'].str.split(',', expand=True)
# 刪除原始欄位
df.drop(columns=['raw_data'], inplace=True)
```

Q1. How many columns are there in this csv file and what are these columns' names?

```
df.head(10)
```

	text	sentiment	source	date/time	user_id	location	confidence_score
0	"I love this product!"	Positive	Twitter	2023-06-15 09:23:14	@user123	New York	0.85
1	"The service was terrible."	Negative	Yelp Reviews	2023-06-15 11:45:32	user456	Los Angeles	0.65
2	"This movie is amazing!"	Positive	IMDb	2023-06-15 14:10:22	moviefan789	London	0.92
3	"I'm so disappointed with their customer suppo...	Negative	Online Forum	2023-06-15 17:35:11	forumuser1	Toronto	0.78
4	"Just had the best meal of my life!"	Positive	TripAdvisor	2023-06-16 08:50:59	foodie22	Paris	0.88
5	"The quality of this product is subpar."	Negative	Amazon Reviews	2023-06-16 10:15:27	shopper123	San Francisco	0.72
6	"I can't stop listening to this song. It's inc...	Positive	Spotify	2023-06-16 13:40:18	musiclover456	Berlin	0.91

然後我利用每個欄位之間都有“，”，來分割不同欄位

(20%) Q2. Do the following steps and show the corresponding results.

- Clean data

尋找資料集中是否有重複的評論，重複的評論有可能是同一個人評的，因此我們會想先把它刪除。

```
#重複值
df.duplicated().sum()
```

21

```
df = df.drop_duplicates()
```

- Drop nan

接下來我們看是否資料中有評論沒有完全的，因為這種數據會對於我們後續的分析有所影響，因此先將他們剔除。

```
#缺失值
```

```
df.isnull().sum()
```

```
text          2
sentiment     2
source        2
date/time     2
user_id       2
location      2
confidence_score  2
dtype: int64
```

```
df = df.dropna(axis=0, how='any')
df.isnull().sum()
```

```
text          0
sentiment     0
source        0
date/time     0
user_id       0
location      0
confidence_score  0
dtype: int64
```

- Convert date and time to datetime

原本 date/time 的 Dtype 是 object，會希望將它轉換成 datetime 格式，方便接下來轉換成年月日的進行。

Convert data and time to datetime

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 75 entries, 0 to 74
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   text                   75 non-null    object
1   sentiment              75 non-null    object
2   source                 75 non-null    object
3   date/time              75 non-null    object
4   user_id                75 non-null    object
5   location               75 non-null    object
6   confidence_score       75 non-null    object
dtypes: object(7)
memory usage: 4.7+ KB
```

```
#將時間調整成python可讀的形式
df['date/time']=pd.to_datetime(df['date/time'])
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 75 entries, 0 to 74
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   text                   75 non-null    object
1   sentiment              75 non-null    object
2   source                 75 non-null    object
3   date/time              75 non-null    datetime64[ns]
4   user_id                75 non-null    object
5   location               75 non-null    object
6   confidence_score       75 non-null    object
dtypes: datetime64[ns](1), object(6)
memory usage: 4.7+ KB
```

- Create new features - month, day, and hour
把月份日期小時分別從 **date/time** 中分割出來。

```
df['month']=df['date/time'].dt.month
df['day']=df['date/time'].dt.day
df['hour']=df['date/time'].dt.hour
df.head()
```

sentiment	source	date/time	user_id	location	confidence_score	month	day	hour
Positive	Twitter	2023-06-15 09:23:14	@user123	New York	0.85	6	15	9
Negative	Yelp Reviews	2023-06-15 11:45:32	user456	Los Angeles	0.65	6	15	11
Positive	IMDb	2023-06-15 14:10:22	moviefan789	London	0.92	6	15	14

- Create new features again - Total Words, Total Chars, and Total Words After Transformation of "Text," where Total Words After Transformation means *The natural logarithm of the word count of the "Text"*.

透過字串的分割與長度的統計等功能，我們能確實的找出題目所需要的這幾個欄位，如下所示。

```
df['Total Words'] = df['text'].str.split().str.len()
df['Total Chars'] = df['text'].str.len()
df['Total Words After Transformation'] = np.log(df['Total Words'])
df.head(10)
```

user_id	location	confidence_score	month	day	hour	Total Words	Total Chars	Total Words After Transformation
@user123	New York	0.85	6	15	9	4	22	1.386294
user456	Los Angeles	0.65	6	15	11	4	27	1.386294
moviefan789	London	0.92	6	15	14	4	24	1.386294

(26%) Q3. Do the following visualizations, eg., histplot, displot, barplot, kdeplot, etc., and write your findings.

1) by Sentiment 'Positive' and 'Negative'

把所有正面評價視為 1，所有負面評價視為 0

並且計算兩者的個數儲存進 Positive 和 Negative 中

然後將數據繪製成圓餅圖

```
print(df['sentiment'].unique())
```

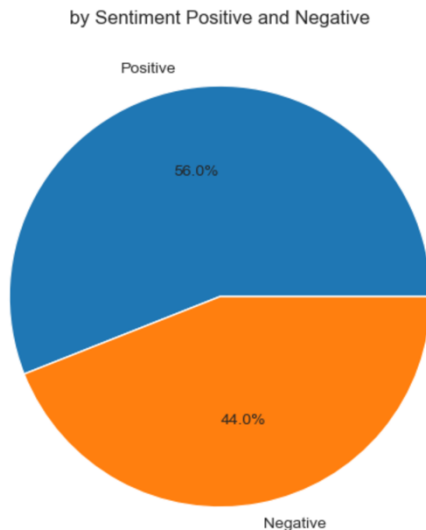
```
[' Positive' ' Negative']
```

```
df = df.dropna(subset=['sentiment'])
df['sentiment_code'] = df['sentiment'].map({' Positive':1, ' Negative':0}).astype('int')
df['sentiment_code']
```

```
0      1
1      0
2      1
3      0
4      1
..
70     1
71     0
72     1
73     1
74     0
Name: sentiment_code, Length: 75, dtype: int32
```

```
Positive=(df['sentiment_code']==1).sum()
Negative=(df['sentiment_code']==0).sum()
```

```
import seaborn as sns
import matplotlib.pyplot as plt
data=[Positive,Negative]
labels=['Positive','Negative']
sns.set_style("whitegrid")
plt.figure(figsize=(6,6))
plt.pie(data, labels=labels, autopct='%1.1f%%')
plt.title('by Sentiment Positive and Negative')
plt.show
```



可以發現正面的評價比負面的來得多。

2) by 'Source' and 'Sentiment'

1. 將正負面評價分開來繪製橫向的長條圖可以比較每一個評論來源的正面評價數量

```
# 設置繪圖風格
sns.set(style="whitegrid")

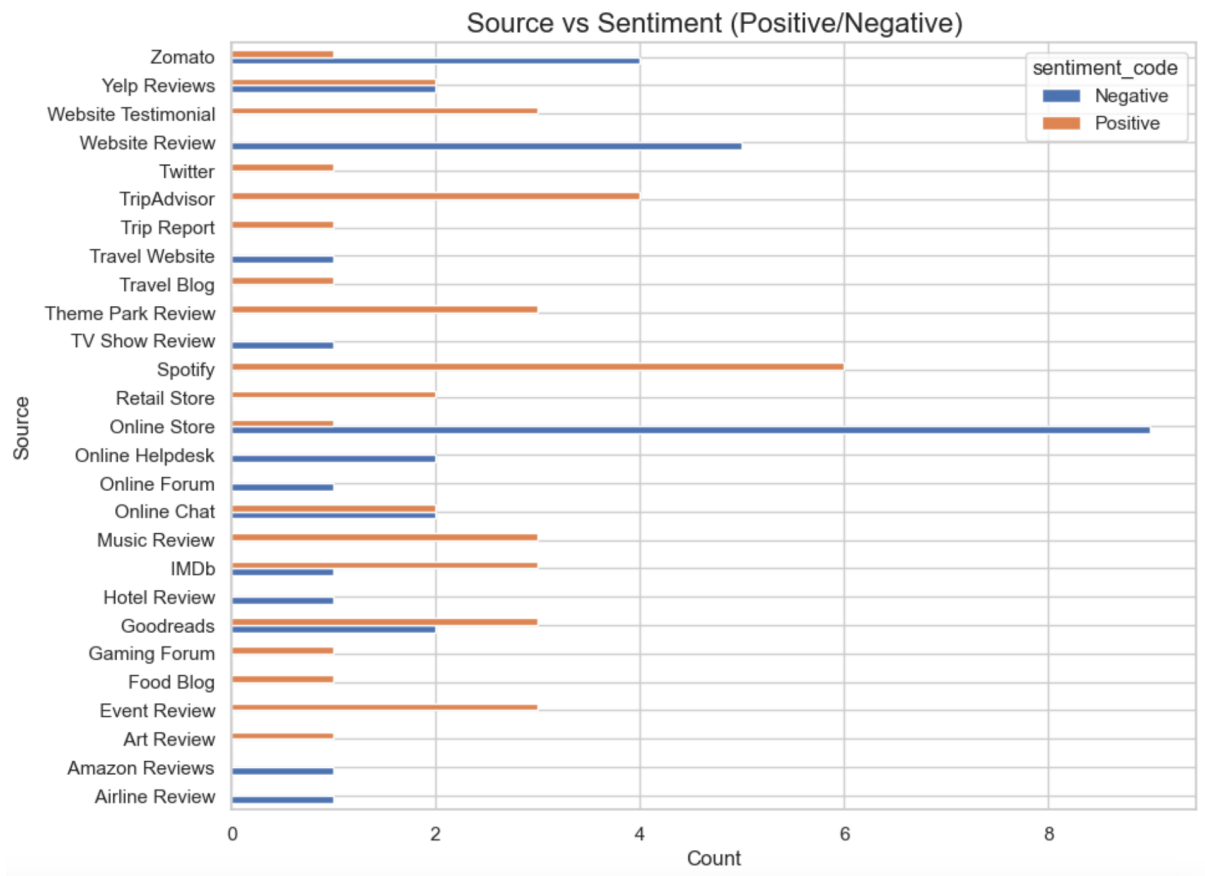
# 使用 groupby 和 size 計算每個 source 中 sentiment 的分佈
source_sentiment_counts = df.groupby(['source', 'sentiment_code']).size().unstack(fill_value=0)

# 替換 sentiment_code 的 0 和 1 為 Negative 和 Positive
source_sentiment_counts.columns = source_sentiment_counts.columns.map({0: 'Negative', 1: 'Positive'})

# 繪製水平條形圖
ax = source_sentiment_counts.plot(kind='barh', stacked=False, figsize=(10, 8))

# 設置標題和標籤
ax.set_title('Source vs Sentiment (Positive/Negative)', fontsize=16)
ax.set_xlabel('Count', fontsize=12)
ax.set_ylabel('Source', fontsize=12)

# 顯示圖表
plt.show()
```



可以發現來自 **online store** 的負面評價最多，而來自 **spotify** 的正面評價最多。
將正負面評價分開來表示的長條圖，更適合分開統計正負面評價的數據，而比較看不出總評論數的些微差距。

2. 另外我也會去了將這負面評價結合在一起的橫向長條圖，我認為這種圖表可以同時去比較各個來源的評論數量，也大致可以統計正面評價的數量，似乎較為直觀。

```
# 設置繪圖風格
sns.set(style="whitegrid")

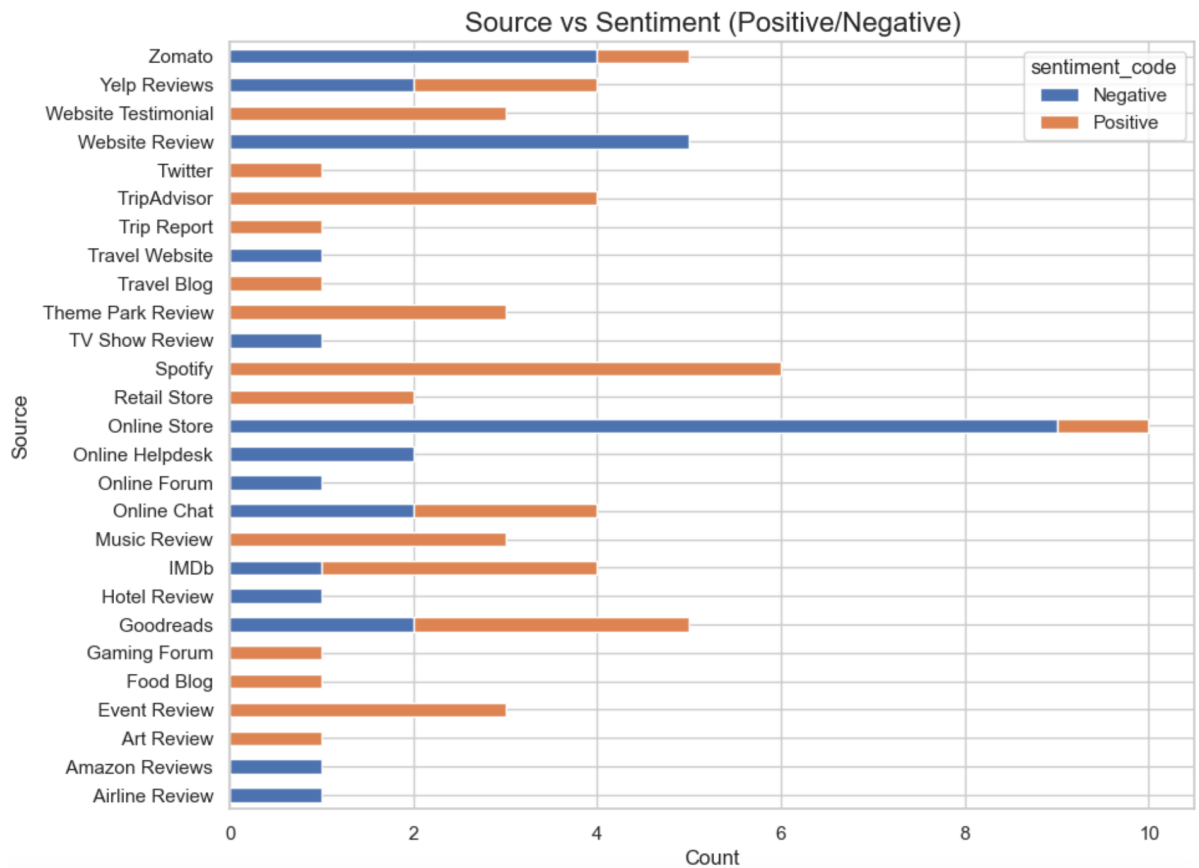
# 使用 groupby 和 size 計算每個 source 中 sentiment 的分佈
source_sentiment_counts = df.groupby(['source', 'sentiment_code']).size().unstack(fill_value=0)

# 替換 sentiment_code 的 0 和 1 為 Negative 和 Positive
source_sentiment_counts.columns = source_sentiment_counts.columns.map({0: 'Negative', 1: 'Positive'})

# 繪製水平條形圖
ax = source_sentiment_counts.plot(kind='barh', stacked=True, figsize=(10, 8))

# 設置標題和標籤
ax.set_title('Source vs Sentiment (Positive/Negative)', fontsize=16)
ax.set_xlabel('Count', fontsize=12)
ax.set_ylabel('Source', fontsize=12)

# 顯示圖表
plt.show()
```



可以看到負面評價最多的 **online store** 評價總數是第一名，而剛剛所提及的 **spotify** 擁有第二多的評價數量，且皆為正面評價，第三名則為正負面評價數量接近的 **goodreads** 拿下。

3) by 'Location' and 'Sentiment'

有了剛剛分析評論來源的經驗，我們在分析地點時，同樣先以堆疊的橫向長條圖來繪製與分別的長條圖來分析。

```
# 設置繪圖風格
sns.set(style="whitegrid")

# 使用 groupby 和 size 計算每個 source 中 sentiment 的分佈
source_sentiment_counts = df.groupby(['location', 'sentiment_code']).size().unstack(fill_value=0)

# 替換 sentiment_code 的 0 和 1 為 Negative 和 Positive
source_sentiment_counts.columns = source_sentiment_counts.columns.map({0: 'Negative', 1: 'Positive'})

# 繪製水平條形圖
ax = source_sentiment_counts.plot(kind='barh', stacked=True, figsize=(10, 8))

# 設置標題和標籤
ax.set_title('Location vs Sentiment (Positive/Negative)', fontsize=16)
ax.set_xlabel('Count', fontsize=12)
ax.set_ylabel('Location', fontsize=12)

# 顯示圖表
plt.show()
```



由上圖可以看出，雪梨的評價數為第一，倫敦則緊追其後，其次是來自巴黎的評價，而前幾名的這幾個地點正面評價皆多於負面評價。

```
# 設置繪圖風格
sns.set(style="whitegrid")

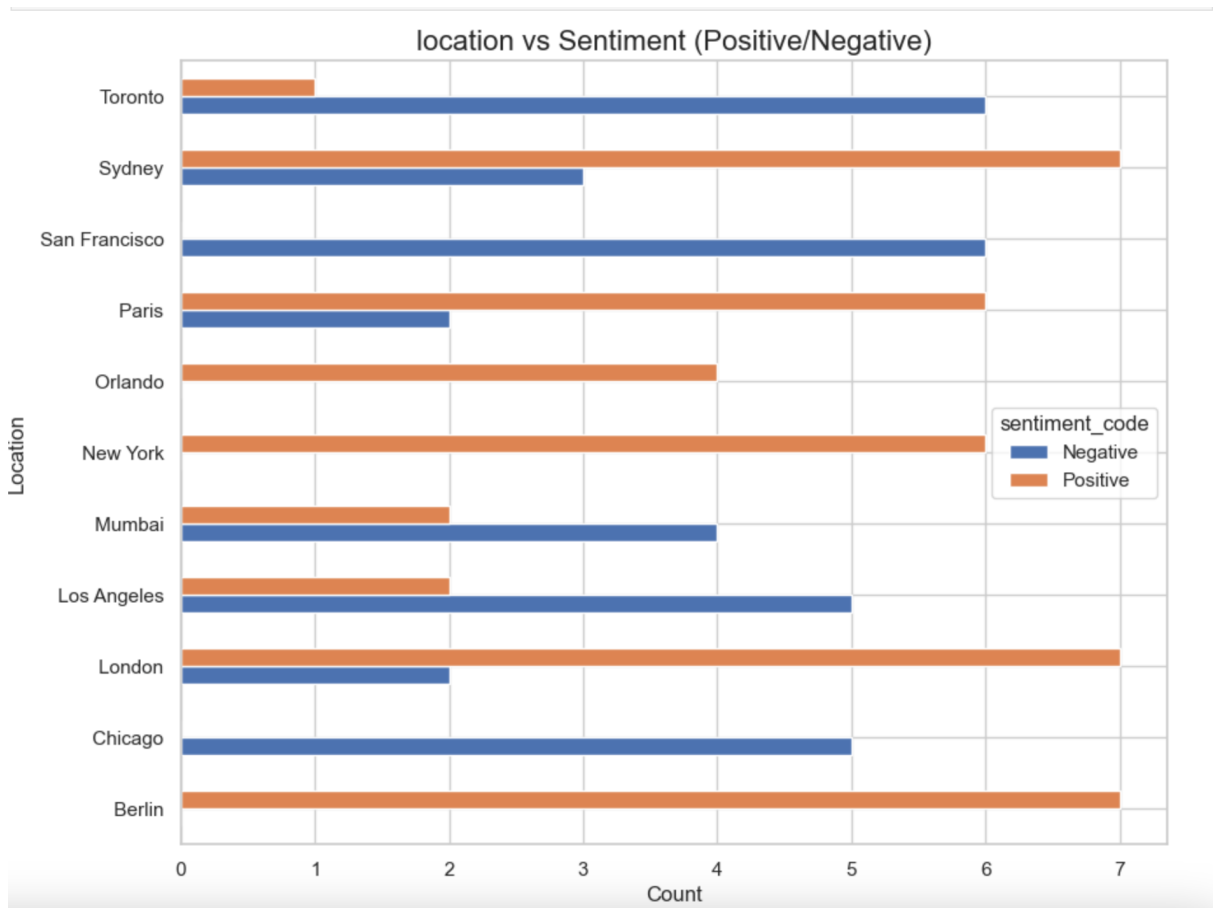
# 使用 groupby 和 size 計算每個 source 中 sentiment 的分佈
source_sentiment_counts = df.groupby(['location', 'sentiment_code']).size().unstack(fill_value=0)

# 替換 sentiment_code 的 0 和 1 為 Negative 和 Positive
source_sentiment_counts.columns = source_sentiment_counts.columns.map({0: 'Negative', 1: 'Positive'})

# 繪製水平條形圖
ax = source_sentiment_counts.plot(kind='barh', stacked=False, figsize=(10, 8))

# 設置標題和標籤
ax.set_title('location vs Sentiment (Positive/Negative)', fontsize=16)
ax.set_xlabel('Count', fontsize=12)
ax.set_ylabel('Location', fontsize=12)

# 顯示圖表
plt.show()
```



我從這張圖看到最明顯的地方是，雪梨倫敦與柏林儘管總評價數是不相同的，但他們的正面評價數量是相同的，非常有趣。

4) by 'Confidence Score' and 'Sentiment'

因為我認為信心指數分佈適合類似人口結構分析的連續之分析，因此根據我在公民課所學我選擇用與人口結構分布圖類似的小提琴圖來進行分析

```
# 設定 Seaborn 風格
sns.set_theme(style="whitegrid")

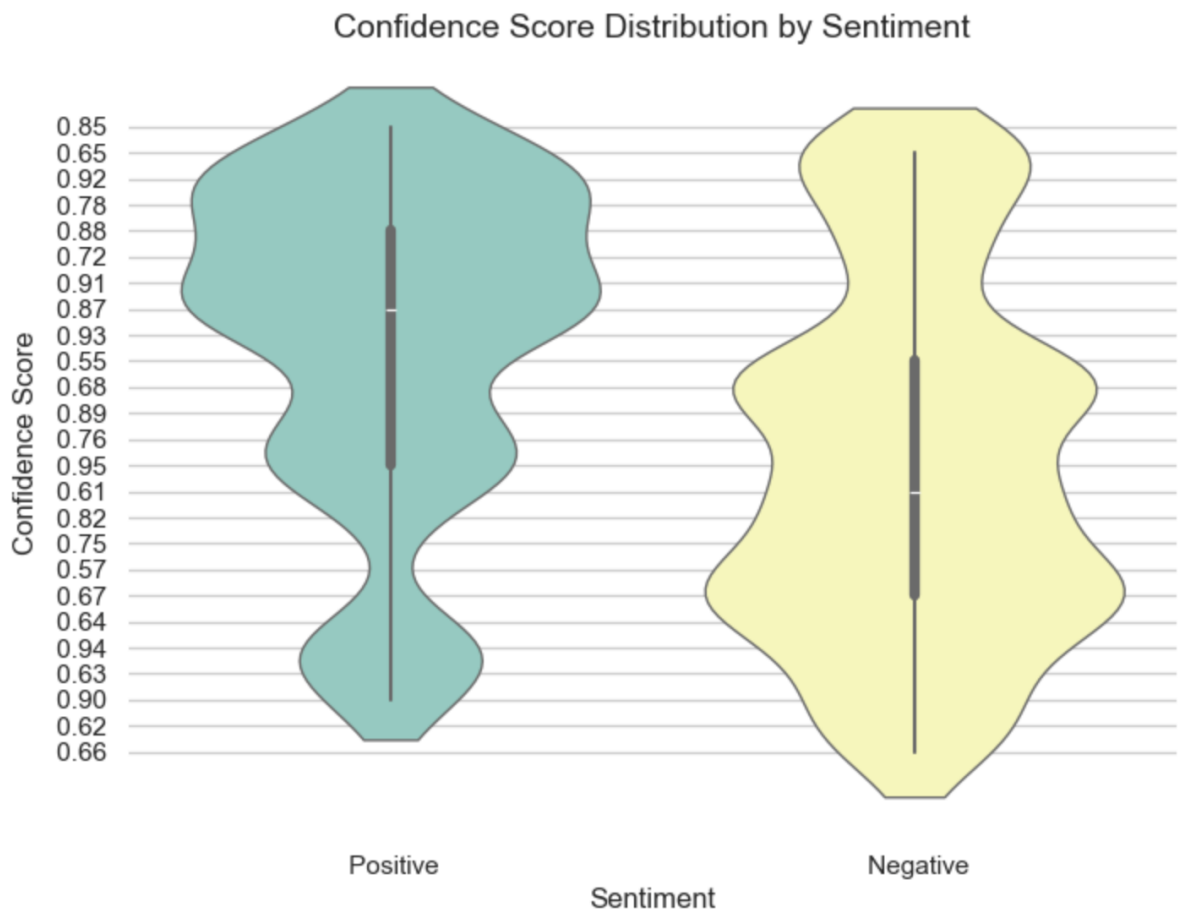
# 設置圖形大小
plt.figure(figsize=(8, 6))

# 繪製小提琴圖
sns.violinplot(x='sentiment', y='confidence_score', data=df, bw_adjust=0.5, cut=1, linewidth=1, palette="Set3")

# 設置標題和標籤
plt.title("Confidence Score Distribution by Sentiment", fontsize=14)
plt.xlabel("Sentiment", fontsize=12)
plt.ylabel("Confidence Score", fontsize=12)

# 移除不必要的邊框
sns.despine(left=True, bottom=True)

# 顯示圖表
plt.show()
```



可以發現大多數信心指數高的都對產品有正面的評價而信心指數低的則有較高的負面評價機率

5) by 'Month' and 'Sentiment'

考慮到月份只有兩個月，因此我們同樣使用長條圖來進行比較

```
# 設置繪圖風格
sns.set(style="whitegrid")

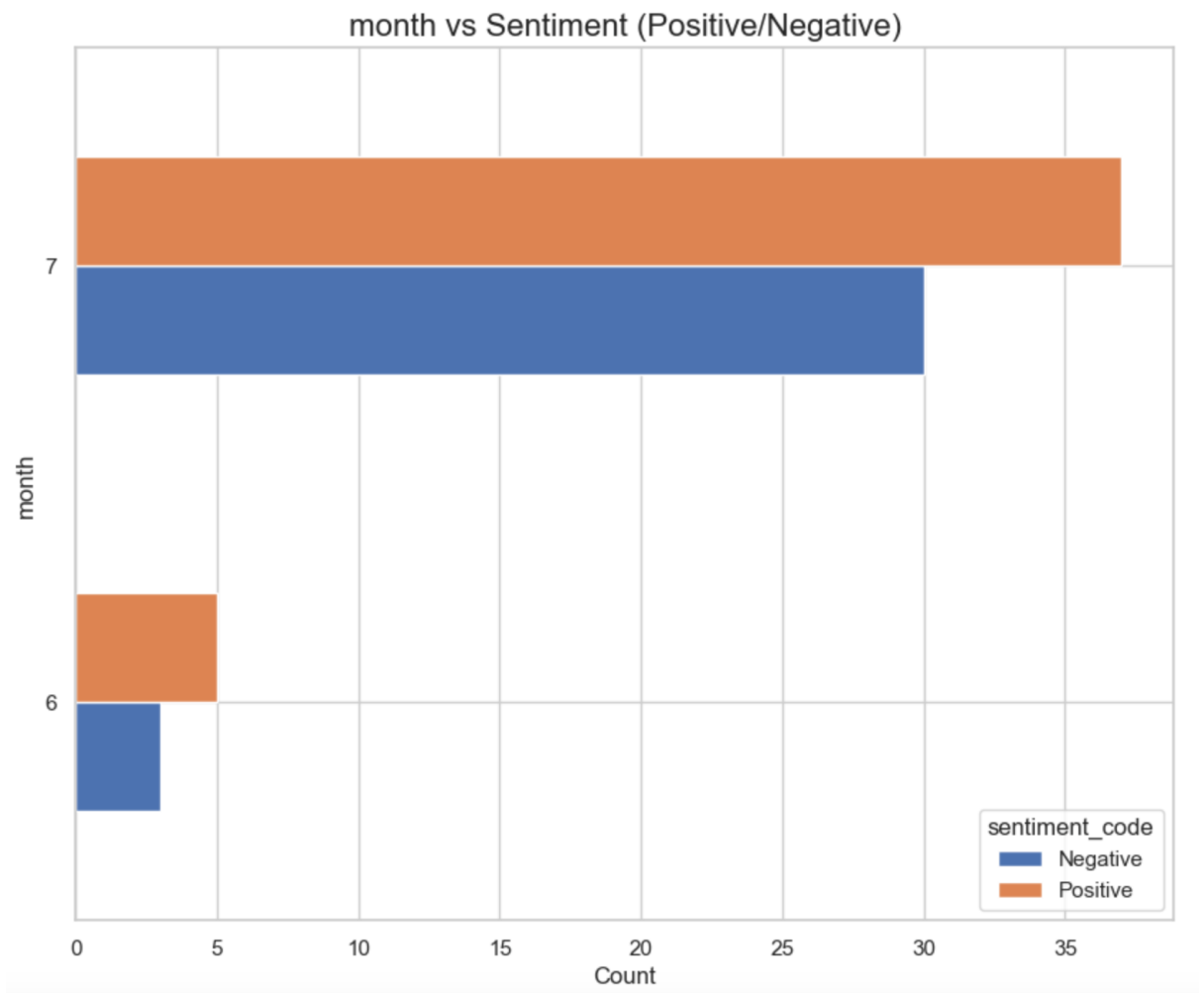
# 使用 groupby 和 size 計算每個 source 中 sentiment 的分佈
source_sentiment_counts = df.groupby(['month', 'sentiment_code']).size().unstack(fill_value=0)

# 替換 sentiment_code 的 0 和 1 為 Negative 和 Positive
source_sentiment_counts.columns = source_sentiment_counts.columns.map({0: 'Negative', 1: 'Positive'})

# 繪製水平條形圖
ax = source_sentiment_counts.plot(kind='barh', stacked=False, figsize=(10, 8))

# 設置標題和標籤
ax.set_title('month vs Sentiment (Positive/Negative)', fontsize=16)
ax.set_xlabel('Count', fontsize=12)
ax.set_ylabel('month', fontsize=12)

# 顯示圖表
plt.show()
```



可以發現七月的評論數量遠比六月來得多
且兩個月份的正面評價都多於負面評價，與第一張圖表 整體的正面評價多於負面評價相符合

6) by 'Day' and 'Sentiment'

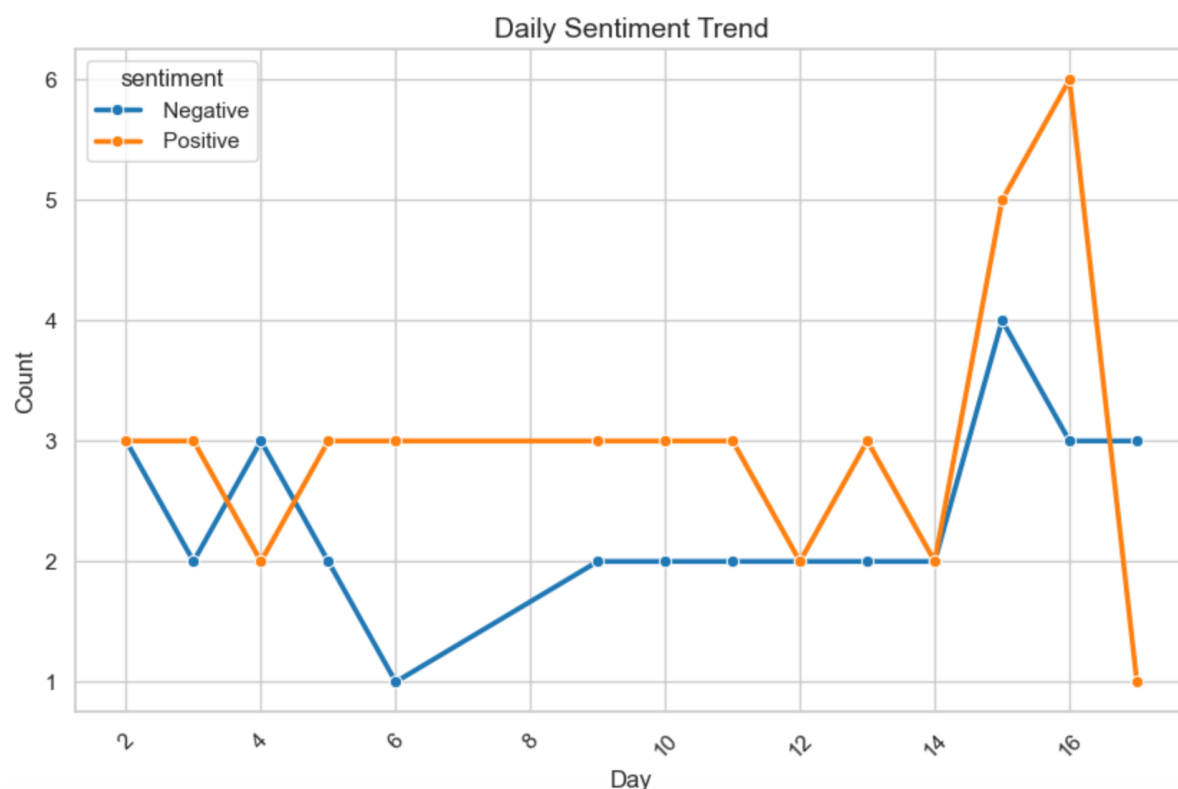
日期的數量就相較月份來得更多了，因此我們可以考慮使用折線圖看。

```
# 計算每日的 sentiment 數量
sentiment_counts = df.groupby(['day', 'sentiment']).size().reset_index(name='count')

# 繪製 Lineplot
plt.figure(figsize=(10, 6))
sns.lineplot(data=sentiment_counts, x="day", y="count", hue="sentiment", marker="o", linewidth=2.5, palette="tab10")

# 設置標題與標籤
plt.title("Daily Sentiment Trend", fontsize=14)
plt.xlabel("Day", fontsize=12)
plt.ylabel("Count", fontsize=12)

# 旋轉 X 軸標籤，使其更易讀
plt.xticks(rotation=45)
```



可以發現接近月中的評價有些微的上升，且正面評價的數量相較於負面評價來得多，我認為這個趨勢觀察兩個月還不夠明顯更適合長期的來觀察。

7) by 'hour' and 'Sentiment'

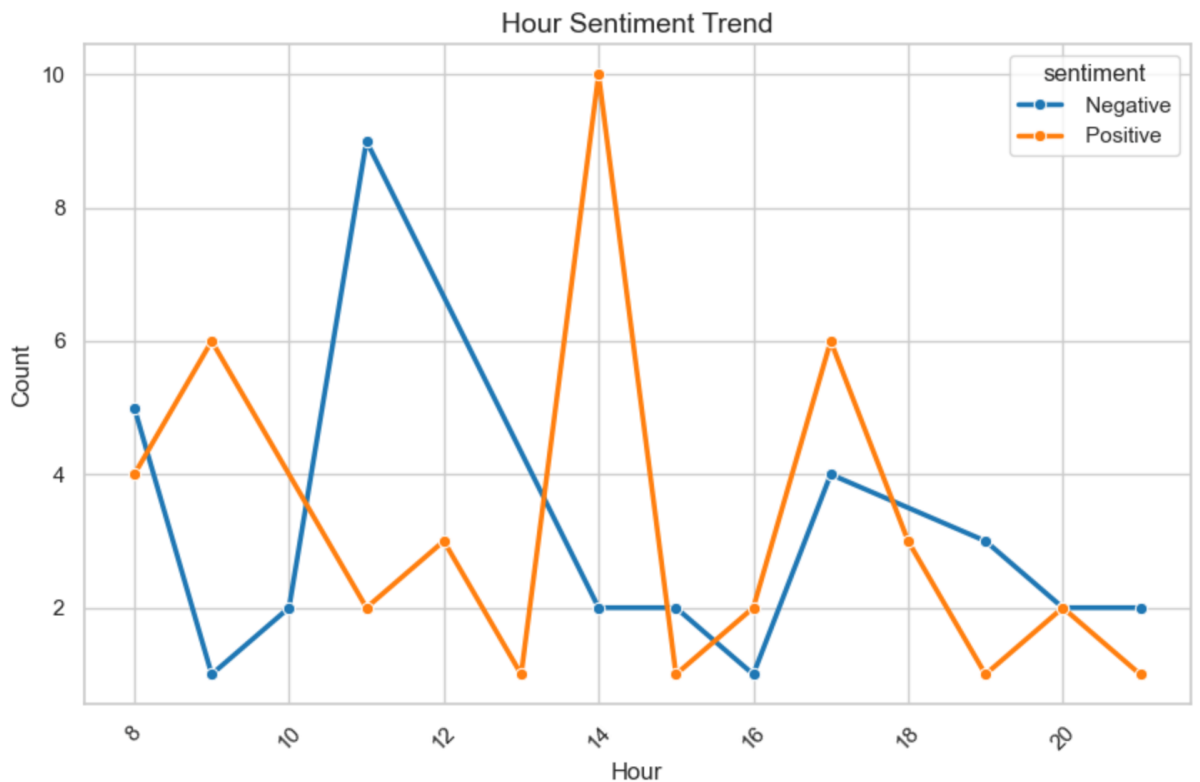
我認為小時來分析的話，同樣是適合使用折線圖的

```
# 計算每小時的 sentiment 數量
sentiment_counts = df.groupby(['hour', 'sentiment']).size().reset_index(name='count')

# 繪製 Lineplot
plt.figure(figsize=(10, 6))
sns.lineplot(data=sentiment_counts, x="hour", y="count", hue="sentiment", marker="o", linewidth=2.5, palette="tab10")

# 設置標題與標籤
plt.title("Hour Sentiment Trend", fontsize=14)
plt.xlabel("Hour", fontsize=12)
plt.ylabel("Count", fontsize=12)

# 旋轉 X 軸標籤，使其更易讀
plt.xticks(rotation=45)
```



可以發現在找成或是晚間負面評價出現的比例有超過正面評價的機會然而在下午時正面評價卻遠多於負面評價也許和人們的評價和作息相關（？）

8) by 'Total Words' and 'Sentiment'

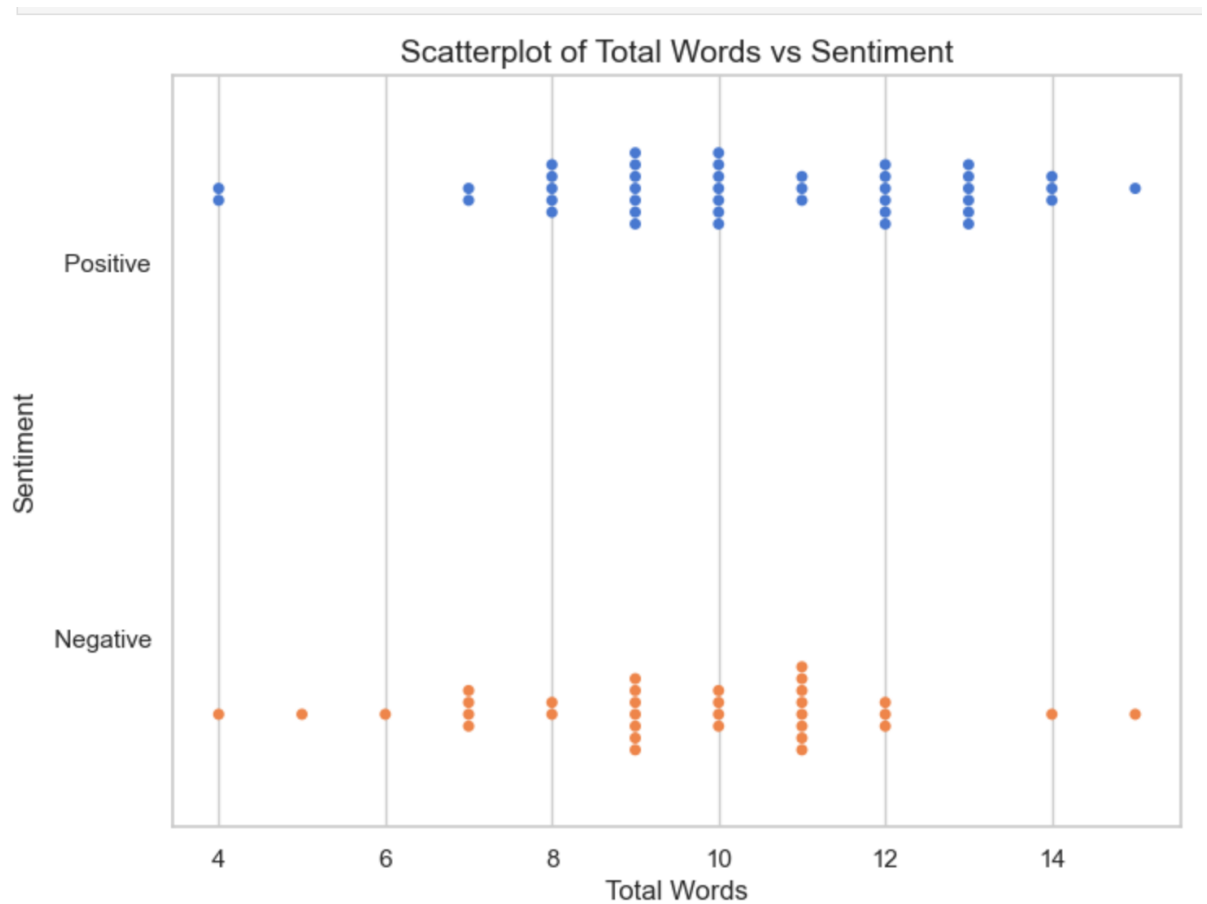
最初的想法是想要使用 **scatterplot** 來分析的，但是顯然 **total words** 的字數並不夠多，似乎不太適合用這個方式

```
# 設定 Seaborn 風格
sns.set_theme(style="whitegrid", palette="muted")

# 繪製 Swarmplot (防止數據點重疊)
plt.figure(figsize=(8, 6))
ax = sns.swarmplot(data=df, x="Total Words", y="sentiment", hue="sentiment", dodge=True)

# 設置標題與標籤
ax.set_title("Scatterplot of Total Words vs Sentiment", fontsize=14)
ax.set_xlabel("Total Words", fontsize=12)
ax.set_ylabel("Sentiment", fontsize=12)

# 顯示圖表
plt.show()
```



於是我改用箱型圖來做圖，這樣的做圖方式可以明顯看到正面評價的字數統計上會比負面評價的字數來得多

```
#Grouped boxplots
# 設定 Seaborn 風格
sns.set_theme(style="ticks", palette="pastel")

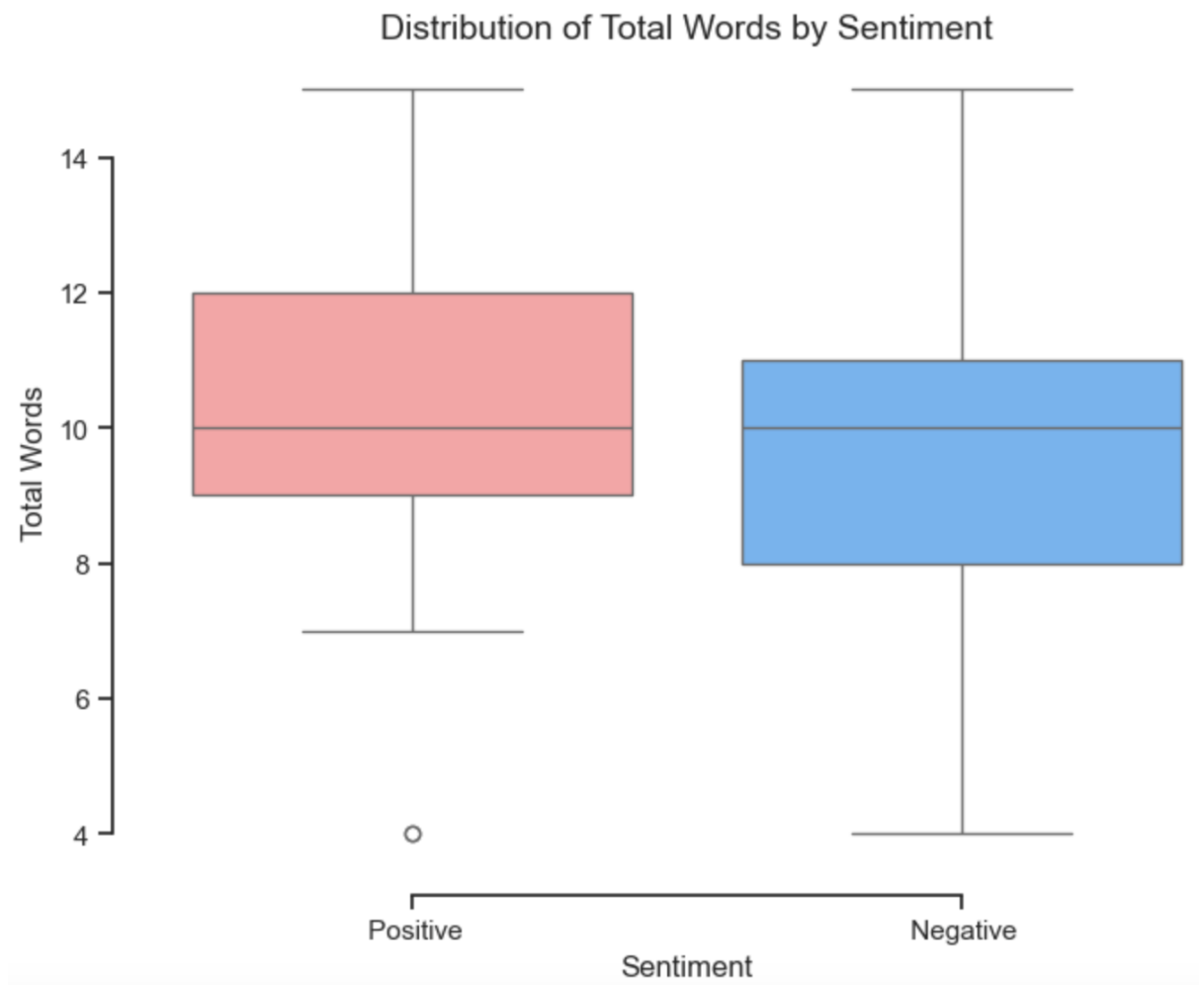
# 設置圖形大小
plt.figure(figsize=(8, 6))

# 繪製 Grouped Boxplot
sns.boxplot(x="sentiment", y="Total Words", data=df, palette=["#FF9999", "#66B3FF"])

# 優化圖表
sns.despine(offset=10, trim=True)

# 設置標題與標籤
plt.title("Distribution of Total Words by Sentiment", fontsize=14)
plt.xlabel("Sentiment", fontsize=12)
plt.ylabel("Total Words", fontsize=12)

# 顯示圖表
plt.show()
```



9) by 'Total Chars' and 'Sentiment'

有了 **total words** 的經驗，分析 **total chars** 時，我同樣使用箱型圖

可以看到正面評價的平均字元數，略為高於負面評價的平均字元數

不過負面評價的資源數分布較為廣，有些人只是情緒性的發表幾個字，而也有人長篇大論的對產品進行批評

```
#Grouped boxplots
# 設定 Seaborn 風格
sns.set_theme(style="ticks", palette="pastel")

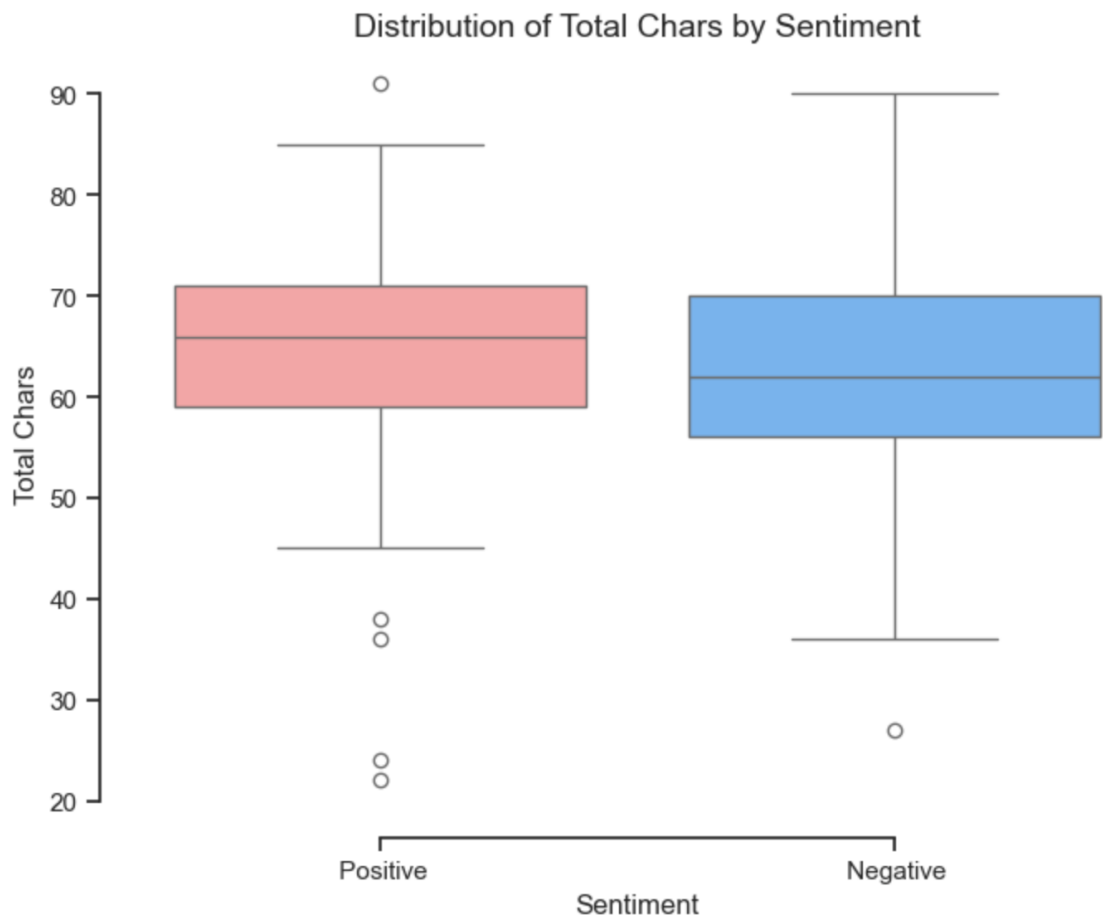
# 設置圖形大小
plt.figure(figsize=(8, 6))

# 繪製 Grouped Boxplot
sns.boxplot(x="sentiment", y="Total Chars", data=df, palette=["#FF9999", "#66B3FF"])

# 優化圖表
sns.despine(offset=10, trim=True)

# 設置標題與標籤
plt.title("Distribution of Total Chars by Sentiment", fontsize=14)
plt.xlabel("Sentiment", fontsize=12)
plt.ylabel("Total Chars", fontsize=12)

# 顯示圖表
plt.show()
```



10) Wordcloud by Sentiment = Negative

文字雲是一種可視化技術，用來顯示一篇或多篇文本中的關鍵詞。具有以下兩個特色：

1. 單詞出現次數越多，字體越大。
2. 可以用不同顏色和形狀來強調某些詞彙。

把負面評價中的文字分別篩選出來放入 worldcloud 的函數中進行運算，再繪製成圖表

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
# 篩選 sentiment = Negative 的文本
negative_text = " ".join(df[df['sentiment'] == 'Negative']['text'])

# 產生文字雲
wordcloud = WordCloud(width=800, height=400, background_color='white', colormap='Reds').generate(negative_text)

# 繪製圖表
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off") # 移除軸線
plt.title("WordCloud for Negative Sentiment", fontsize=14)
plt.show()
```

由圖可以看出，**product** 和 **terrible** 是最常出現在負面評價裡的文字



11) Wordcloud by Sentiment = Positive

把正面評價中的文字分別篩選出來放入 worldcloud 的函數中進行運算，再繪製成圖表


```

from collections import Counter
import nltk
from nltk.corpus import stopwords
import re

# 確保你已下載 nltk 的 stopwords
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

# 將 'im' 加入到停用詞中，因為測試時發現這個無意義的詞混入其中
stop_words.add('im')

# 篩選 Negative Sentiment 的文本
negative_texts = df[df['sentiment'].str.lower() == 'negative']['text'].dropna().astype(str)

# 轉換成單一大文本
all_negative_text = " ".join(negative_texts)

# 清理文本：移除標點符號、數字，並轉為小寫
all_negative_text = re.sub(r"[^a-zA-Z\s]", "", all_negative_text).lower()

# 分詞 (Tokenization)
words = all_negative_text.split()

# 過濾掉停用詞
filtered_words = [word for word in words if word not in stop_words]

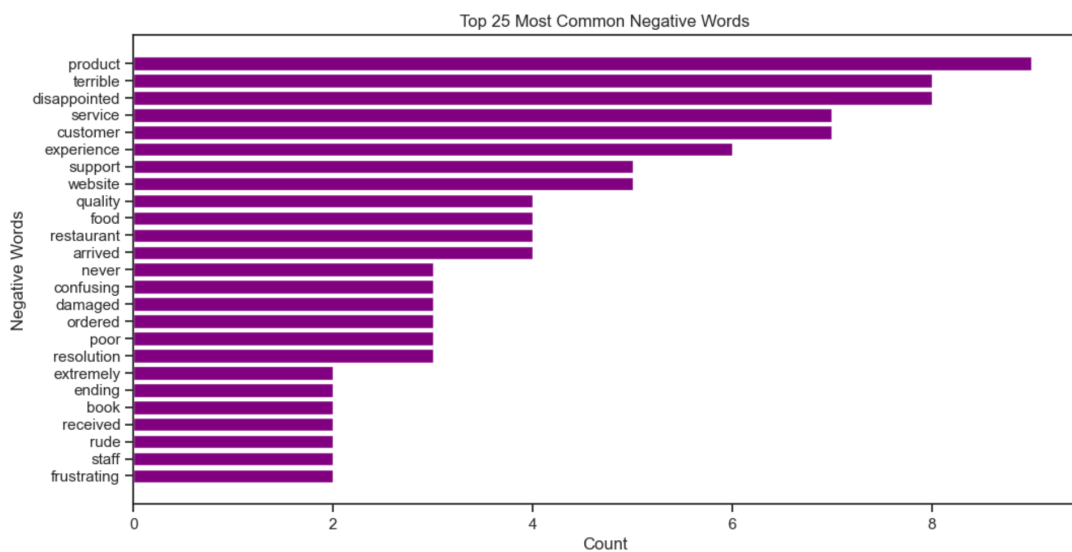
# 計算詞頻
word_counts = Counter(filtered_words)

# 取得最常見的 25 個詞
top_negative_words = word_counts.most_common(25)

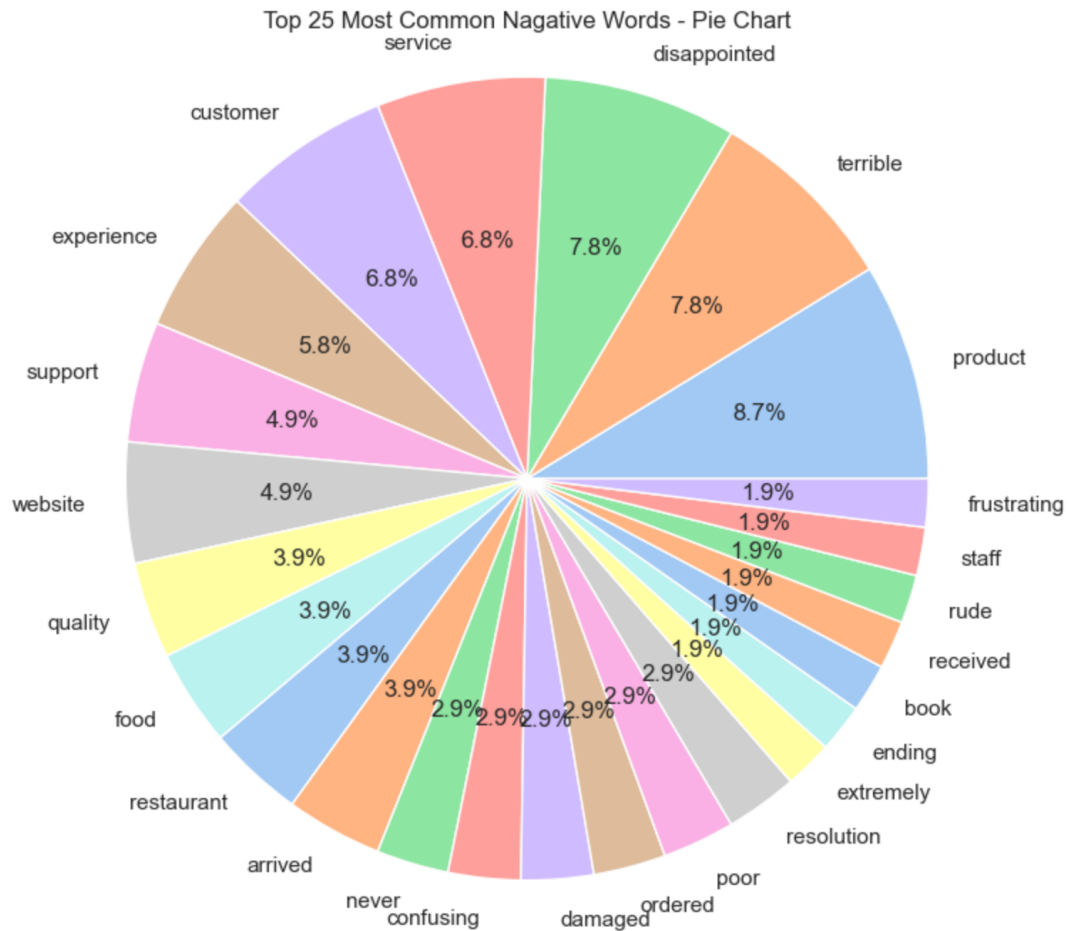
# 轉為 DataFrame 以便視覺化
top_words_df = pd.DataFrame(top_negative_words, columns=['Word', 'Count'])

# 繪製條形圖
plt.figure(figsize=(12, 6))
plt.barh(top_words_df['Word'], top_words_df['Count'], color='purple')
plt.xlabel("Count")
plt.ylabel("Negative Words")
plt.title("Top 25 Most Common Negative Words")
plt.gca().invert_yaxis() # 讓最高頻的詞彙排在上方
plt.show()

```



```
# 繪製圓餅圖
plt.figure(figsize=(8, 8))
plt.pie(top_words_df['Count'], labels=top_words_df['Word'], autopct='%1.1f%%')
plt.title("Top 25 Most Common Negative Words - Pie Chart")
plt.axis("equal") # 讓圓餅圖看起來像圓形
plt.show()
```



13) by Top 25 Positive Words

將正面的詞彙選出來並移除其中的雜訊包括停用詞、標點符號等，然後把他們個別拆分去計算出現的個數，將最常見的前 25 個拿出來繪製長條圖以及圓餅圖。

可以發現相較於負面詞彙，正面詞彙各詞彙出現的次數差異其實並不大。

```

# 確保你已下載 nltk 的 stopwords
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

# 將 'im' 加入到停用詞中，因為測試時發現這個無意義的詞混入其中
stop_words.add('im')
stop_words.add('cant')
# 篩選 Positive Sentiment 的文本
positive_texts = df[df['sentiment'].str.lower() == 'positive']['text'].dropna().astype(str)

# 轉換成單一大文本
all_positive_text = " ".join(positive_texts)

# 清理文本：移除標點符號、數字，並轉為小寫
all_positive_text = re.sub(r"[^a-zA-Z\s]", "", all_positive_text).lower()

# 分詞 (Tokenization)
words = all_positive_text.split()

# 過濾掉停用詞
filtered_words = [word for word in words if word not in stop_words]

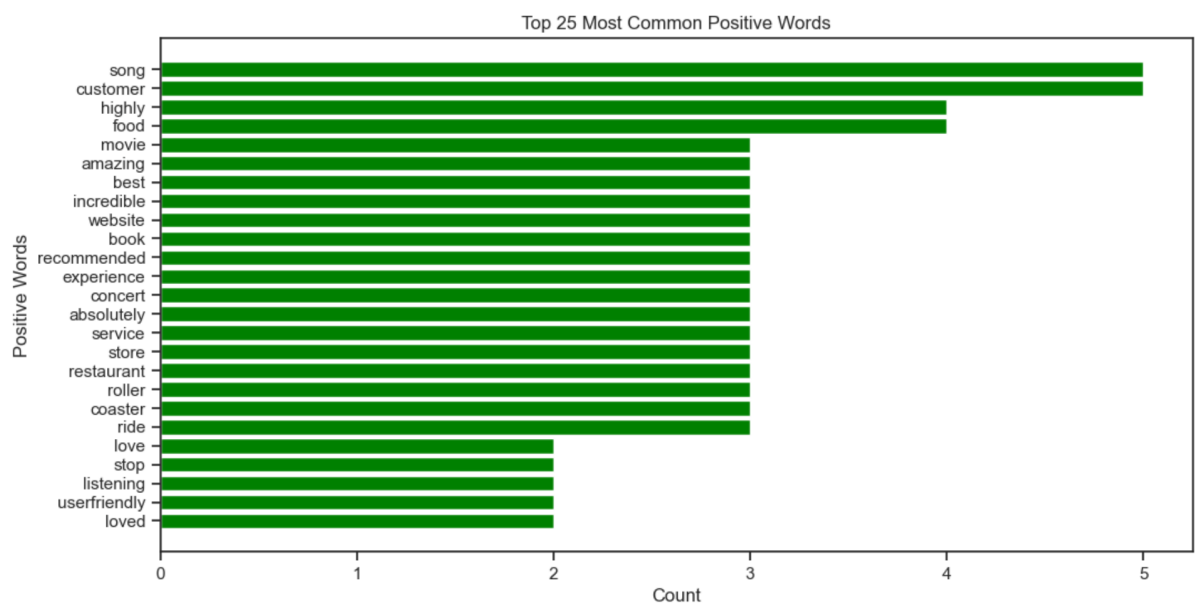
# 計算詞頻
word_counts = Counter(filtered_words)

# 取得最常見的 25 個詞
top_positive_words = word_counts.most_common(25)

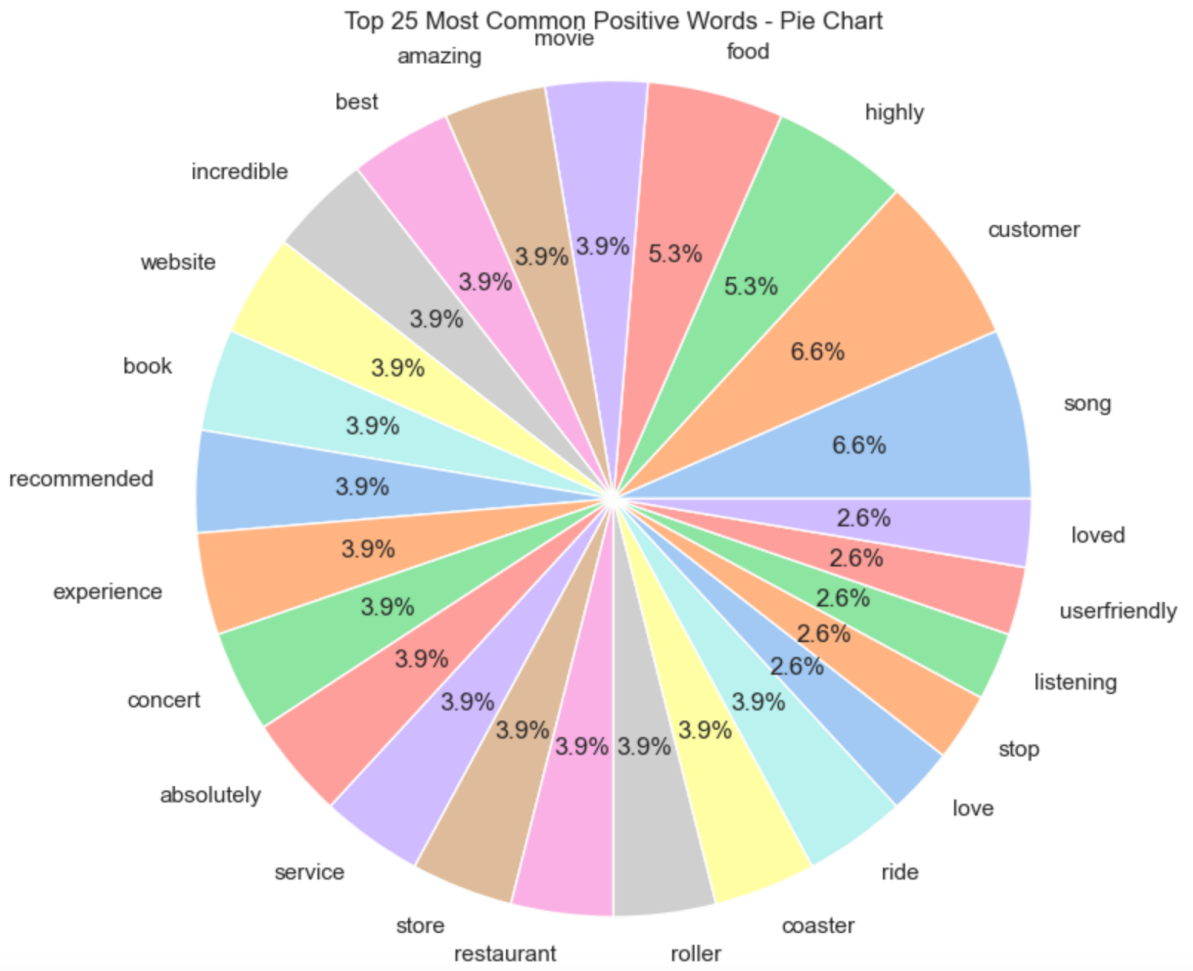
# 轉為 DataFrame 以便視覺化
top_words_df = pd.DataFrame(top_positive_words, columns=['Word', 'Count'])

# 繪製條形圖
plt.figure(figsize=(12, 6))
plt.barh(top_words_df['Word'], top_words_df['Count'], color='green')
plt.xlabel("Count")
plt.ylabel("Positive Words")
plt.title("Top 25 Most Common Positive Words")
plt.gca().invert_yaxis() # 讓最高頻的詞彙排在上方
plt.show()

```



```
# 繪製圓餅圖
plt.figure(figsize=(8, 8))
plt.pie(top_words_df['Count'], labels=top_words_df['Word'], autopct='%1.1f%%')
plt.title("Top 25 Most Common Positive Words - Pie Chart")
plt.axis("equal") # 讓圓餅圖看起來像圓形
plt.show()
```



AI 的使用

HW2 與 AI 的對話在下面連結

<https://chatgpt.com/share/67efd4cb-d6bc-800d-bc4a-2819066a4f52>

基本上我問 ChatGPT 的問題是圖表的繪製，因為這對我來說是不熟悉的語法以及後面文字雲的部分整個是對我來說是新的觀念，我覺得對我幫助蠻大的。

<https://claude.ai/share/05aa3d05-6806-439c-b0c2-4719d516a7e1>

我也有交錯著使用 **claude.ai**，偏向問較為知識或觀念的問題和少數程式撰寫